



**IE4030**  
**Virtualization and Cloud Computing**  
**Technologies**  
**4<sup>th</sup> Year, 1<sup>st</sup> Semester**

**Assignment 02**

**Use Docker for the deployment of essential network services.**

Submitted to

Sri Lanka Institute of Information Technology

In partial fulfilment of the requirements for the  
Bachelor of Science Special Honors Degree in Information Technology

2026.10.05

## **Declaration**

I certify that this report does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief, it does not contain any material previously published or written by another person, except where due reference is made in text.

Registration Number: IT22014122

Name: S S Hewapathirana

## Executive Summary

This report presents the successful completion of Assignment 02, which focuses on deploying essential network services using Docker containers. The objective was to build, configure, and test multiple containerized services including DHCP, DNS, and Proxy (Squid) within a Linux environment to gain hands-on experience in container-based system administration and network configuration.

The assignment was carried out on an Ubuntu 22.04 host system with Docker installed. Each service was developed in separate parts, starting with the creation of Dockerfiles to automate the image-building process, followed by container deployment and functionality testing. The DHCP container was configured to dynamically allocate IP addresses within the local subnet, the DNS container handled domain name resolution, and the Squid Proxy container managed caching and HTTP traffic control.

Additionally, all custom-built Docker images were tagged and pushed to Docker Hub under the user repository *sawera2001*, ensuring remote availability and version control. The verification phase confirmed that all containers were running successfully, providing stable and responsive services.

Overall, this project demonstrates the practical application of Docker for network service virtualization, highlighting containerization's benefits such as isolation, portability, scalability, and simplified management. Through this assignment, a comprehensive understanding of deploying and managing network infrastructure services in a Dockerized environment was achieved.

## Table of Contents

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>01. Introduction</b>                                      | <b>5</b>  |
| <b>02. System Requirements and Environment</b>               | <b>6</b>  |
| <b>03. Part A – Initial Setup</b>                            | <b>8</b>  |
| <b>3.1. Installing Docker</b>                                | <b>8</b>  |
| 01. Updating System Packages                                 | 8         |
| 02. Installing Docker and Compose Plugin                     | 8         |
| 03. Creating GPG Keyring Directory                           | 8         |
| 04. Adding Docker GPG Key                                    | 8         |
| 05. Adding Docker Repository to APT Sources                  | 9         |
| 06. Resolving GPG Key Error and Updating Repository          | 9         |
| 07. Installing Docker Engine and Dependencies                | 10        |
| 08. Enabling Docker Service and Configuring User Permissions | 10        |
| 09. Verifying Docker Installation                            | 10        |
| 10. Successful Docker Installation Confirmation.             | 11        |
| <b>3.2 Creating Docker Hub Account</b>                       | <b>12</b> |
| 01. Creating a New Docker Hub Account                        | 12        |
| 02. Docker Hub Dashboard (Successful Login)                  | 12        |
| 03. Docker Device Activation Page                            | 13        |
| 04. Device Activation Successful                             | 13        |
| 05. Docker CLI Login Confirmation                            | 14        |
| <b>04. Part B – DNS Server</b>                               | <b>15</b> |
| <b>4.1 Docker file Creation</b>                              | <b>15</b> |
| 01. Creating the Dockerfile                                  | 15        |
| 02. Docker file content (part 1)                             | 15        |
| 03. Complete Dockerfile code                                 | 15        |
| 04. Creating named.conf                                      | 16        |
| 05. Displaying named.conf content                            | 16        |
| 08. Creating named.conf.options                              | 17        |
| 09. Verifying named.conf.options content                     | 17        |
| 10. Folder view with three files                             | 17        |
| <b>4.2 Building the DNS Image</b>                            | <b>18</b> |
| 01. Creating the DNS zone database file (db.csne.vcct.com)   | 18        |
| 02. Verifying db.csne.vcct.com                               | 19        |
| 03. Folder view with all configuration files                 | 19        |

|                                                         |           |
|---------------------------------------------------------|-----------|
| 04. Building the Docker image (complete build log)..... | 20        |
| <b>4.3 Running the DNS Container.....</b>               | <b>20</b> |
| <b>4.4 Testing DNS Resolution.....</b>                  | <b>21</b> |
| <b>05. Part C – DHCP Server .....</b>                   | <b>22</b> |
| <b>5.1 Dockerfile Creation .....</b>                    | <b>22</b> |
| 01. Creating the DHCP directory .....                   | 22        |
| 02. Writing the Dockerfile.....                         | 22        |
| 03. Verifying Dockerfile content .....                  | 22        |
| 04. Creating dhcpd.conf file .....                      | 22        |
| 05. Verifying dhcpd.conf configuration .....            | 23        |
| 06. Folder verification .....                           | 23        |
| <b>5.2 Building the DHCP Image .....</b>                | <b>24</b> |
| <b>5.3 Running the DHCP Container.....</b>              | <b>25</b> |
| 01. Verifying dhcpd.conf configuration .....            | 25        |
| 02. Checking running containers.....                    | 25        |
| 03. Viewing DHCP logsViewing DHCP logs .....            | 25        |
| <b>5.4 Testing DHCP Functionality .....</b>             | <b>26</b> |
| <b>06. Part D – Proxy Server (Squid).....</b>           | <b>27</b> |
| <b>6.1 Dockerfile Creation .....</b>                    | <b>27</b> |
| 01. Creating the Squid directory .....                  | 27        |
| 02. Writing the Dockerfile.....                         | 27        |
| 03. Verifying the Dockerfile file creation .....        | 27        |
| 04. Creating the Squid configuration file .....         | 28        |
| <b>6.2 Building the Proxy Image .....</b>               | <b>30</b> |
| 01. Building the image with Docker .....                | 30        |
| <b>6.3 Running the Proxy Container .....</b>            | <b>30</b> |
| 01. Running the Squid container.....                    | 30        |
| 01. Verifying container status .....                    | 31        |
| <b>6.4 Testing Proxy and Caching.....</b>               | <b>31</b> |
| 01. Testing web access via proxy .....                  | 31        |
| <b>07. Part E – Docker Hub .....</b>                    | <b>32</b> |
| <b>7.1 Pushing Images to Docker Hub .....</b>           | <b>32</b> |
| 01. Logging in to Docker Hub .....                      | 32        |
| 02. Listing local images .....                          | 32        |
| 03. Pushing images to Docker Hub .....                  | 33        |

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>7.2 Pulling Images and Redeployment.....</b>       | <b>34</b> |
| 01. Removing and re-pulling DNS image.....            | 34        |
| 02. Redeploying DHCP server .....                     | 34        |
| 03. Redeploying Proxy server .....                    | 35        |
| <b>08. Part F – Verification .....</b>                | <b>36</b> |
| <b>8.1 Using docker ps to Verify Services .....</b>   | <b>36</b> |
| <b>8.1 Final Validation.....</b>                      | <b>36</b> |
| <b>09. Client Testing and Verification.....</b>       | <b>37</b> |
| <b>9.1 DHCP Client Configuration .....</b>            | <b>37</b> |
| <b>9.2 DNS Resolution Test .....</b>                  | <b>37</b> |
| <b>9.3 Proxy Server Functionality .....</b>           | <b>38</b> |
| <b>9.3 Summary of Client-Side Validation .....</b>    | <b>39</b> |
| <b>10. Bonus Task – Docker Compose .....</b>          | <b>40</b> |
| <b>10.1 Pulling Images and Redeployment.....</b>      | <b>40</b> |
| 01. Updating System Packages .....                    | 40        |
| 02. Installing Docker Compose.....                    | 40        |
| 03. Verifying Docker Compose Installation.....        | 41        |
| 04. Writing docker-compose.yml.....                   | 41        |
| 05. Verifying the Compose File .....                  | 43        |
| 06. Preparing DHCP Lease Directory.....               | 43        |
| <b>10.2 Running Services via Docker Compose .....</b> | <b>44</b> |
| 02. Starting Containers .....                         | 44        |
| 02. Checking Container Status .....                   | 44        |
| 03. Viewing DHCP Service Logs .....                   | 44        |
| 04. DNS Service Verification.....                     | 45        |
| 05. Proxy Service Verification .....                  | 46        |
| <b>11. Conclusion .....</b>                           | <b>47</b> |
| <b>12. References.....</b>                            | <b>48</b> |

# 01. Introduction

## 1.1 Background

Virtualization and containerization are fundamental technologies in modern computing environments. Containers, such as those provided by Docker, allow developers and system engineers to deploy applications in isolated environments with minimal overhead. Unlike traditional virtualization that relies on hypervisors and virtual machines, containers share the host operating system kernel, resulting in faster deployment, better resource utilization, and simplified scaling.

In networking, services such as DNS, DHCP, and Proxy servers are critical for managing communication between clients and servers. Deploying these services within containers provides portability, consistency, and easier management across multiple environments. This assignment focuses on applying Docker to set up and manage these essential network services.

## 1.2 Objectives

The main objectives of this assignment are:

- To gain hands-on experience with Docker as a containerization platform.
- To design and deploy network services (DNS, DHCP, and Proxy) inside Docker containers.
- To practice building Docker images using Docker files and uploading them to Docker Hub for sharing and redeployment.
- To evaluate the configuration and performance of virtualized network services in a containerized environment.
- To develop skills in documenting and verifying deployments through testing, screenshots, and logs.

## 02. System Requirements and Environment

### 2.1 Hardware Requirements

To perform this assignment, a computer system with sufficient resources was required to run Docker containers efficiently. The following hardware specifications were used:

- **Processor (CPU):** Dual-core or higher (Intel/AMD)
- **Memory (RAM):** Minimum 4 GB (8 GB recommended for smoother container operation)
- **Storage:** At least 20 GB of free disk space for Docker images, containers, and logs

#### Network Adapters:

- NAT adapter (for internet connectivity)
- Host-only adapter (for local container communication)

### 2.2 Software Requirements

The assignment was carried out using the following software components:

- **Operating System:** Ubuntu 22.04 LTS (Linux environment is recommended for Docker)
- **Docker Engine:** Docker CE (Community Edition) latest stable version
- **Docker Compose (Optional):** Installed for managing multiple services simultaneously
- **Text Editor:** nano / vim / Visual Studio Code for editing Dockerfiles and configuration files
- **Testing Tools:**
  - nslookup or dig for verifying DNS resolution
  - ifconfig/ip addr for DHCP testing
  - Web browser (Firefox/Chrome) configured to use Proxy for Squid testing

## 2.3 Environment Setup

- Docker was installed using the official repository packages to ensure the latest stable release.
- Docker Hub account was created for image upload and sharing.
- A local network (**192.168.8.0/24**) was used for testing DNS, DHCP, and Proxy functionality.
- Containers were allocated limited resources ( $\leq 1$  CPU, 512 MB RAM for the Proxy server) to simulate lightweight deployments.
- Screenshots were taken at each stage to verify successful installation, configuration, and execution.

## 03. Part A – Initial Setup

### 3.1. Installing Docker

#### 01. Updating System Packages

The package list on the Ubuntu system was updated using the *apt update* command to ensure that all repositories and package information were up to date before installing Docker.

```
sawera@VCCT: ~  
sawera@VCCT:~$ sudo apt update  
[sudo] password for sawera:  
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease  
Hit:2 http://lk.archive.ubuntu.com/ubuntu noble InRelease  
Hit:3 http://lk.archive.ubuntu.com/ubuntu noble-updates InRelease  
Hit:4 http://lk.archive.ubuntu.com/ubuntu noble-backports InRelease  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
52 packages can be upgraded. Run 'apt list --upgradable' to see them.  
sawera@VCCT:~$
```

#### 02. Installing Docker and Compose Plugin

*sudo apt install -y docker.io docker-compose-plugin*

Docker and Docker Compose plugin were installed using the apt package manager. The -y flag automatically confirmed installation prompts.

```
sawera@VCCT:~$ sudo apt install -y docker.io docker-compose-plugin
```

#### 03. Creating GPG Keyring Directory

A secure directory */etc/apt/keyrings* was created to store GPG keys, which are used to verify the authenticity of software packages from external repositories.

```
sawera@VCCT:~$ sudo install -m 0755 -d /etc/apt/keyrings
```

#### 04. Adding Docker GPG Key

The official Docker GPG key was downloaded and stored in the keyring directory. This key ensures that Docker packages come from a trusted source.

```
sawera@VCCT:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg
--dearmor -o /etc/apt/keyrings/docker.gpg

File '/etc/apt/keyrings/docker.gpg' exists.
Overwrite? (y/N)
Enter new filename:
gpg: no valid OpenPGP data found.
gpg: dearmoring failed: File exists
sawera@VCCT:~$ sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

## 05. Adding Docker Repository to APT Sources

The official Docker repository was added to the Ubuntu sources list, allowing the system to retrieve Docker packages directly from Docker's maintained repository.

```
sawera@VCCT:~$ echo \
> "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.g
pg] \
> https://download.docker.com/linux/ubuntu \
> $(. /etc/os-release; echo $VERSION_CODENAME) stable" \
> | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sawera@VCCT:~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.8 kB]
Hit:2 http://lk.archive.ubuntu.com/ubuntu noble InRelease
Err:1 https://download.docker.com/linux/ubuntu noble InRelease
  The following signatures couldn't be verified because the public key is not av
ailable: NO_PUBKEY 7EA0A9C3F273FCD8
Hit:3 http://lk.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://lk.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
W: GPG error: https://download.docker.com/linux/ubuntu noble InRelease: The foll
owing signatures couldn't be verified because the public key is not available: N
O_PUBKEY 7EA0A9C3F273FCD8
E: The repository 'https://download.docker.com/linux/ubuntu noble InRelease' is
not signed.
N: Updating from such a repository can't be done securely, and is therefore disa
bled by default.
```

## 06. Resolving GPG Key Error and Updating Repository

A retry was made to fix a GPG verification issue by redownloading the Docker GPG key and updating the repository successfully.

```
sawera@VCCT:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg
--dearmor -o /etc/apt/keyrings/docker.gpg
File '/etc/apt/keyrings/docker.gpg' exists. Overwrite? (y/N)
```

## 07. Installing Docker Engine and Dependencies

The full Docker suite was installed, including the Docker Engine (*docker-ce*), command-line interface (*docker-ce-cli*), container runtime (*containerd.io*), and build tools (*buildx*, *docker-compose*).

```
sawera@VCCT:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
File '/etc/apt/keyrings/docker.gpg' exists. Overwrite? (y/N) y
sawera@VCCT:~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.8 kB]
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://lk.archive.ubuntu.com/ubuntu noble InRelease
Get:4 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [30.9 kB]
Hit:5 http://lk.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:6 http://lk.archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 79.7 kB in 3s (25.5 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
52 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

## 08. Enabling Docker Service and Configuring User Permissions

The Docker service was enabled to start automatically on system boot and was immediately started.

```
sudo usermod -aG docker $USER
```

The current user was added to the Docker group to run Docker commands without using *sudo*.

```
sawera@VCCT:~$ sudo systemctl enable --now docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
sawera@VCCT:~$ sudo usermod -aG docker $USER
```

## 09. Verifying Docker Installation

Docker version and Docker Compose version were verified successfully.

Running the hello-world container confirmed that Docker was correctly installed and able to run containers.

```
sawera@VCCT:~$ newgrp docker
sawera@VCCT:~$ docker --version
Docker version 28.4.0, build d8eb465
sawera@VCCT:~$ docker compose version
Docker Compose version v2.39.4
sawera@VCCT:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:54e66cc1dd1fcb1c3c58bd8017914dbed8701e2d8c74d9262e26bd9cc1642d31
```

## 10. Successful Docker Installation Confirmation.

The output confirmed successful installation and communication between the Docker client and daemon. It also demonstrated that the system could pull images from Docker Hub and run containers.

```
sawera@VCCT:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:54e66cc1dd1fcb1c3c58bd8017914dbed8701e2d8c74d9262e26bd9cc1642d31
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

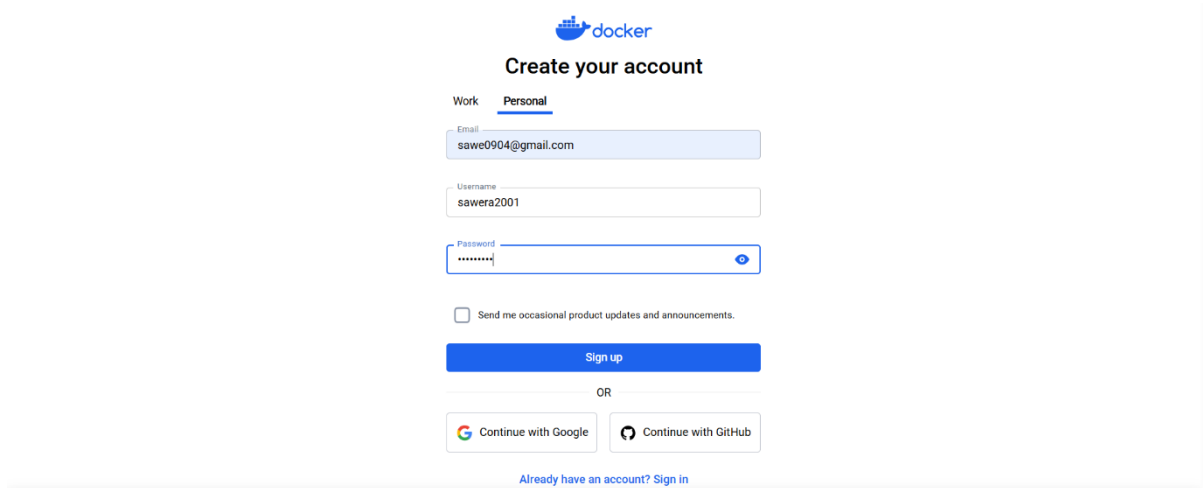
To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
```

## 3.2 Creating Docker Hub Account

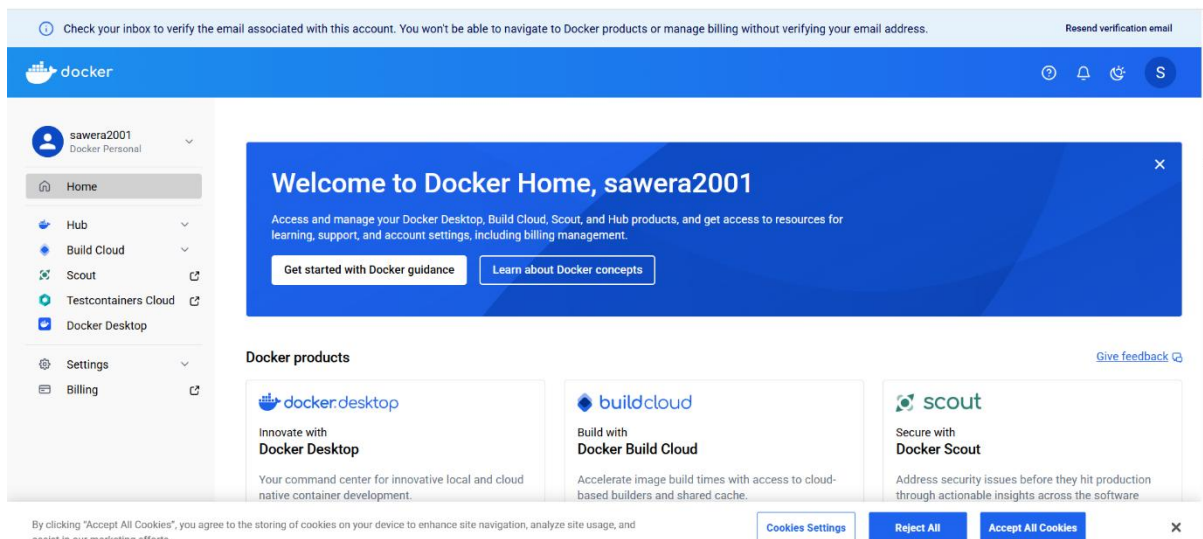
### 01. Creating a New Docker Hub Account

The Docker Hub account creation page is shown. Here, a new personal account was created by entering an email, username, and password, which will be used to log in and manage Docker images online.



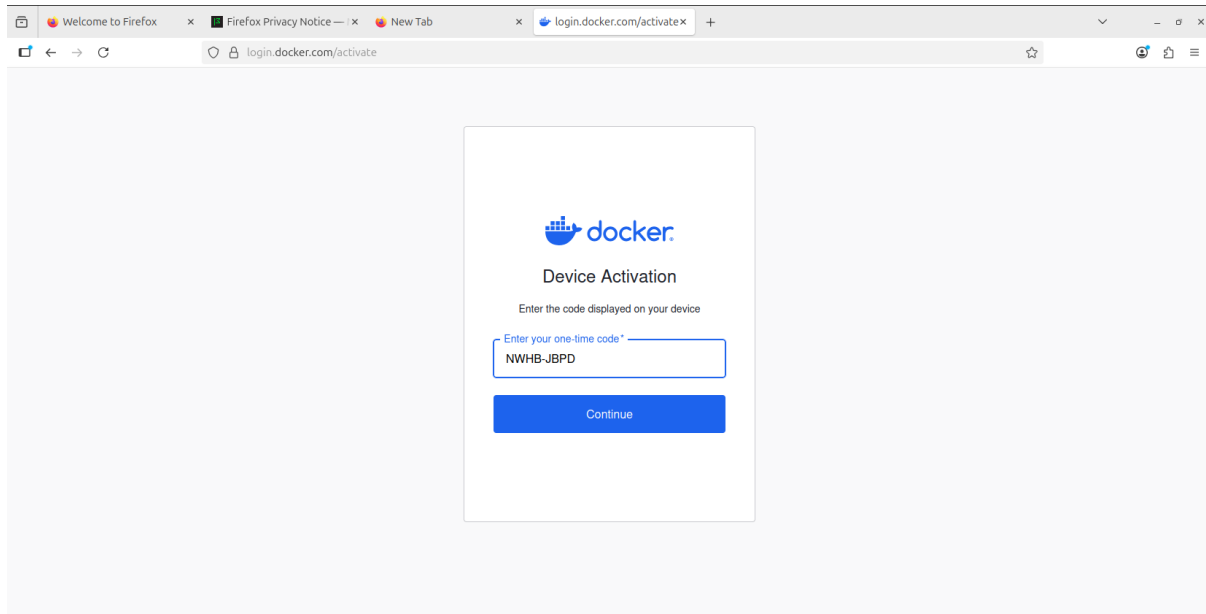
### 02. Docker Hub Dashboard (Successful Login)

After account creation, the Docker Hub dashboard was displayed, confirming successful login. This dashboard allows users to manage repositories, images, and Docker-related services.



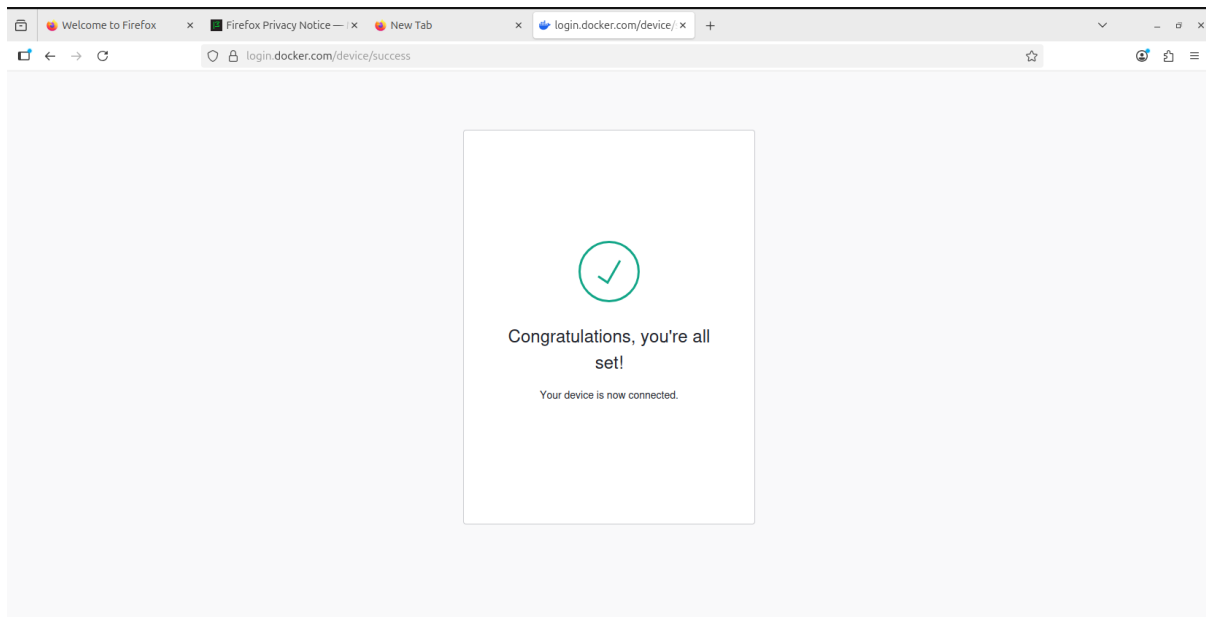
### 03. Docker Device Activation Page

When logging in from the terminal, Docker provided a one-time code for device verification. The user entered this code on the Docker activation page to authenticate the system securely.



### 04. Device Activation Successful

The confirmation page shows that the device was successfully linked to the Docker Hub account, allowing command-line access for pushing and pulling images.



## 05. Docker CLI Login Confirmation

The docker login command was used in the terminal to authenticate the local Docker client with Docker Hub. The successful login message confirms that the system is now connected to the Docker Hub account.

```
root@VCCT:/home/sawera# docker login

USING WEB-BASED LOGIN

i Info → To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: TPNX-XHSK
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser...

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@VCCT:/home/sawera#
```

## 04. Part B – DNS Server

### 4.1 Docker file Creation

#### 01. Creating the Dockerfile

Created a new Dockerfile inside the dns folder. This file defines all the steps required to build a container image for my DNS server.

```
root@VCCT:~/assignment02# docker rm -f dns 2>/dev/null || true
root@VCCT:~/assignment02# docker run -d --name dns -p 192.168.8.128:53:53/udp -p 192.168.8.128:53:53/tcp --restart unless-stopped sawera2001/dns-bind9:1.0
93885d8d39b6c6f754c70fd2f9f86678586bf7d241f84dc427c5cf04bbfdb204
root@VCCT:~/assignment02# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                                                 NA
93885d8d39b6   sawera2001/dns-bind9:1.0           "named -g -c /etc/bi..." About a minute ago Up About a minute 192.168.8.128:53->53/tcp, 192.168.8.128:53->53/udp dn
MES
root@VCCT:~/assignment02# dig @192.168.8.128 csne.vcct.com A

;<<> DiG 9.18.30.0ubuntu0.24.04.2-Ubuntu <<> @192.168.8.128 csne.vcct.com A
(1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 24312
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags: udp: 1232
;; COOKIE: 11906627d1f7fc310100000068d423e00f5d02ba86d52671 (good)
;; QUESTION SECTION:
;; csne.vcct.com.                IN      A
;; ANSWER SECTION:
csne.vcct.com.                3600    IN      A      192.168.8.132
;; Query time: 1 msec
;; SERVER: 192.168.8.128#53(192.168.8.128) (UDP)
;; WHEN: Wed Sep 24 22:31:20 +0530 2025
;; MSG SIZE rcvd: 86

root@VCCT:~/assignment02#
```

#### 02. Docker file content (part 1)

The FROM ubuntu:24.04 base image and a RUN command to install the required DNS packages (bind9 and dnsutils) while cleaning the package cache. These commands prepare the container to run a DNS server.

```
root@VCCT:~/assignment02/dns# cat > Dockerfile <<'EOF'
> FROM ubuntu:24.04
> RUN apt-get update && apt-get install -y bind9 dnsutils && rm -rf /var/lib/apt
/lists/*
> COPY named.conf /etc/bind/named.conf
> COPY named.conf.options /etc/bind/named.conf.options
> COPY db.csne.vcct.com /etc/bind/db.csne.vcct.com
> EXPOSE 53/udp 53/tcp
> CMD ["named", "-g", "-c", "/etc/bind/named.conf"]
> EOF
```

#### 03. Complete Dockerfile code

This shows the full Dockerfile with all instructions:

- COPY statements to move named.conf, named.conf.options, and the zone file into /etc/bind/ inside the container.
- EXPOSE 53/udp 53/tcp to make DNS ports accessible.

- The CMD line runs the BIND9 service using the main configuration file.

```
root@VCCT:~/assignment02/dns# cat Dockerfile
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y bind9 dnsutils && rm -rf /var/lib/apt/l
ists/*
```

#### 04. Creating named.conf

The named.conf file that defines server-wide options such as the directory for cache, query permissions (allow-query { 192.168.8.0/24; }), and the DNS zone csne.vcct.com. This file tells BIND which zones to serve and where to find their database files.

```
root@VCCT:~/assignment02/dns# cat > named.conf <<'EOF'
> options {
>     directory "/var/cache/bind";
>     recursion no;
>     allow-query { 192.168.8.0/24; };
>     listen-on { any; };
>     listen-on-v6 { none; };
>     dnssec-validation no;
> };
> zone "csne.vcct.com" {
>     type master;
>     file "/etc/bind/db.csne.vcct.com";
> };
E> EOF
```

#### 05. Displaying named.conf content

Reopened the named.conf to verify it was saved correctly with all options and the zone "csne.vcct.com" block referencing /etc/bind/db.csne.vcct.com.

```
root@VCCT:~/assignment02/dns# cat named.conf
options {
    directory "/var/cache/bind";
    recursion no;
    allow-query { 192.168.8.0/24; };
    listen-on { any; };
    listen-on-v6 { none; };
    dnssec-validation no;
};
zone "csne.vcct.com" {
    type master;
    file "/etc/bind/db.csne.vcct.com";
};
root@VCCT:~/assignment02/dns#
```

## 08. Creating named.conf.options

This screenshot shows the heredoc creation of the **named.conf.options** file. You defined an access control list (ACL) localnet for the 192.168.8.0/24 network and specified general options (directory, recursion off, and query policy). It limits who can use your DNS server.

```
root@VCCT:~/assignment02/dns# cat > named.conf.options <<'EOF'
> acl localnet { 192.168.8.0/24; };
> options {
>     directory "/var/cache/bind";
>     recursion no;
>     allow-query { localnet; };
>     listen-on { any; };
>     dnssec-validation no;
> };
> EOF
```

## 09. Verifying named.conf.options content

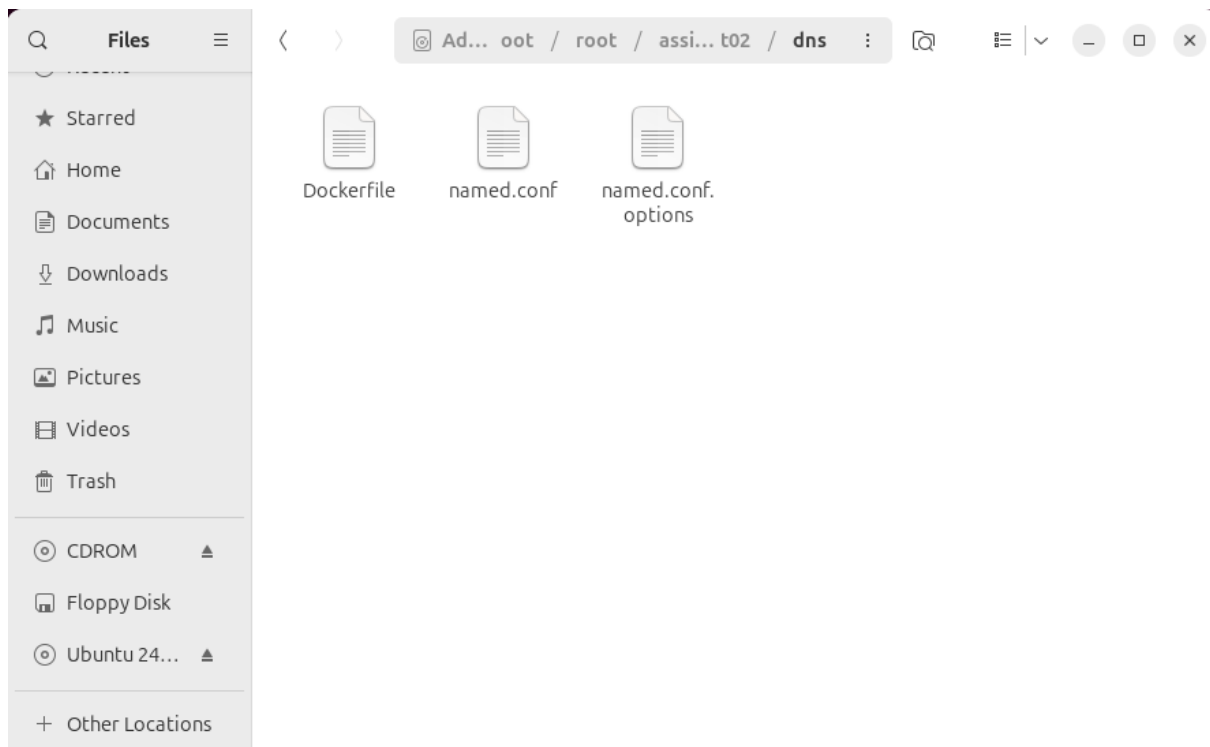
```
root@VCCT:~/assignment02/dns# cat named.conf.options
acl localnet { 192.168.8.0/24; };
options {
    directory "/var/cache/bind";
    recursion no;
    allow-query { localnet; };
    listen-on { any; };
    dnssec-validation no;
};
root@VCCT:~/assignment02/dns#
```

## 10. Folder view with three files

Now the directory contains all three key configuration files:

[Dockerfile](#), [named.conf](#), and [named.conf.options](#).

This confirms that the environment setup for DNS configuration is complete.



## 4.2 Building the DNS Image

### 01. Creating the DNS zone database file (db.csne.vcct.com)

This shows how you defined the forward-zone database file. It specifies the SOA (Start of Authority) record and A records mapping csne.vcct.com → 192.168.8.132 and ns1.csne.vcct.com → 192.168.8.132. This tells DNS how to resolve the domain.

```
root@VCCT:~/assignment02/dns# cat > db.csne.vcct.com <<'EOF'
> $TTL 1H
> @ IN SOA ns1.csne.vcct.com. admin.csne.vcct.com. (
>     20250924 ; serial
>     1H      ; refresh
>     15M     ; retry
>     1W      ; expire
>     1H )    ; minimum
> IN NS  ns1.csne.vcct.com.
> ns1 IN A  192.168.8.132
> @ IN A   192.168.8.132
> EOF
```

## 02. Verifying db.csne.vcct.com

Displayed the file content again to double-check that all records and syntax were correct.

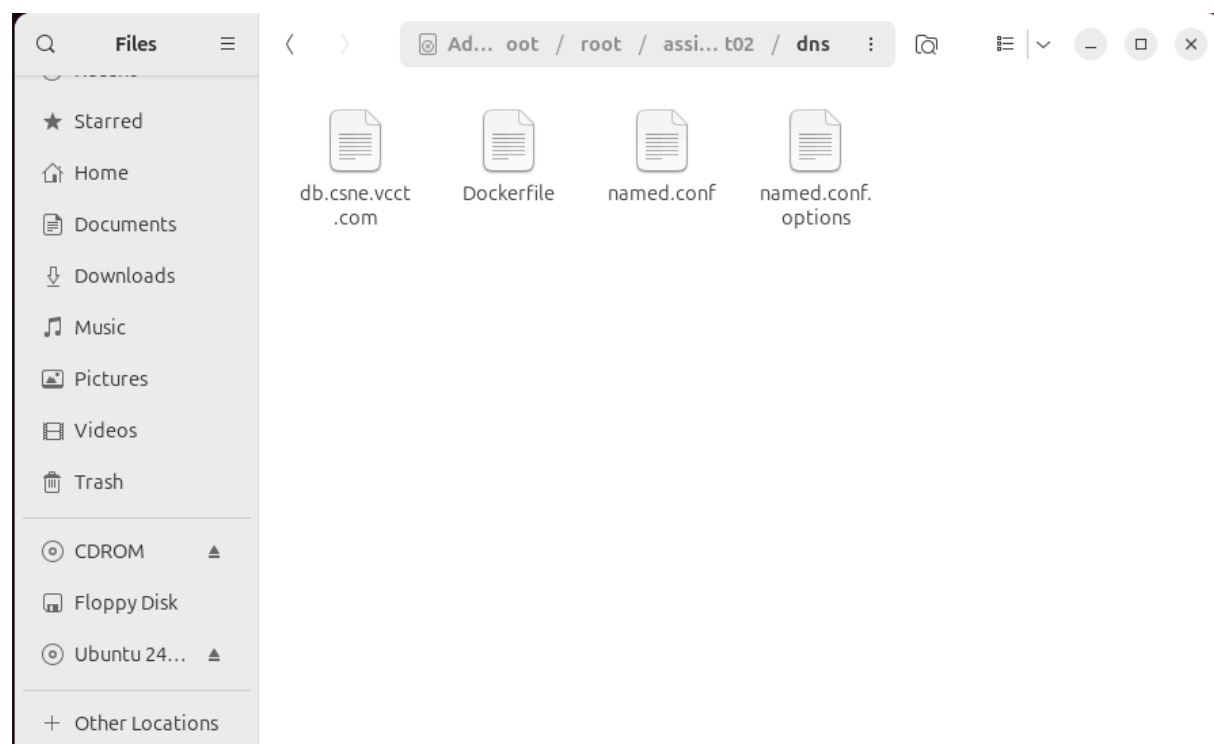
```
root@VCCT:~/assignment02/dns# cat db.csne.vcct.com
$TTL 1H
@ IN SOA ns1.csne.vcct.com. admin.csne.vcct.com. (
    20250924 ; serial
    1H      ; refresh
    15M     ; retry
```

## 03. Folder view with all configuration files

The folder now shows four files:

[Dockerfile](#), [named.conf](#), [named.conf.options](#), and [db.csne.vcct.com](#).

This confirms that every file referenced in the Dockerfile exists and is ready for building.



#### 04. Building the Docker image (complete build log)

Docker executed all steps from the Dockerfile: installing BIND9, copying files, and exporting the final image. The “FINISHED” message at the bottom means the image was successfully built.

```
root@VCCT:~/assignment02/dns# docker build -t sawera2001/dns-bind9:1.0 .
[+] Building 16.6s (5/10)                                docker:default
=> [internal] load build definition from Dockerfile      0.1s
=> => transferring dockerfile: 351B                      0.0s
=> [internal] load metadata for docker.io/library/ubuntu:24.04 10.2s
=> [auth] library/ubuntu:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 2B                            0.0s
=> [1/5] FROM docker.io/library/ubuntu:24.04@sha256:353675e2a41babd526e2 6.2s
=> => resolve docker.io/library/ubuntu:24.04@sha256:353675e2a41babd526e2 0.1s
=> => sha256:985be7c735afdf6f18aaa122c23f87d989c30bba4e9aa24 424B / 424B 0.0s
=> => sha256:6d79abd4c96299aa91f5a4a46551042407568a3858b 2.30kB / 2.30kB 0.0s
=> => sha256:953cdd4133718b72c5d0a78e754c1405c02510fdb5 6.29MB / 29.72MB 6.1s
=> => sha256:353675e2a41babd526e2b837d7ec780c2a05bca0164 6.69kB / 6.69kB 0.0s
=> [internal] load build context                        0.1s
=> => transferring context: 847B                          0.0s
```

#### 4.3 Running the DNS Container

The docker ps output shows the container is running and listening on both ports 53/tcp and 53/udp. Then, the dig query confirmed the containerized DNS server responds correctly to name resolution requests.

```
root@VCCT:~/assignment02# docker rm -f dns 2>/dev/null || true
root@VCCT:~/assignment02# docker run -d --name dns -p 192.168.8.128:53:53/udp -p 192.168.8.128:53:53/tcp --restart unless-stopped sawera2001/dns-bind9:1.0
93885d8d39b6c6f754c70fd2f9f86678586bf7d241f84dc427c5cf04bbfdb204
root@VCCT:~/assignment02#
```

```
root@VCCT:~/assignment02# docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED        STATUS        PORTS
93885d8d39b6   sawera2001/dns-bind9:1.0 "named -g -c /etc/bi..." About a minute ago Up About a minute 192.168.8.128:53->53/tcp, 192.168.8.128:53->53/udp
root@VCCT:~/assignment02#
```

#### 4.4 Testing DNS Resolution

The output returned a valid A record showing csne.vcct.com → 192.168.8.132 with status NOERROR. This proved that the Dockerized DNS server was working correctly and resolving domain names on the local network.

```
root@VCCT: ~/assignment02
; <<>> DiG 9.18.30-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.8.132 csne.vcct.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 28855
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 1232
;; COOKIE: 80a020a651216b1d0100000068e107404685d9dcbf12ac79 (good)
;; QUESTION SECTION:
;csne.vcct.com.                IN      A

;; ANSWER SECTION:
csne.vcct.com.                3600    IN      A      192.168.8.132

;; Query time: 1 msec
;; SERVER: 192.168.8.132#53(192.168.8.132) (UDP)
;; WHEN: Sat Oct 04 17:08:40 +0530 2025
;; MSG SIZE rcvd: 86
```

## 05. Part C – DHCP Server

### 5.1 Dockerfile Creation

#### 01. Creating the DHCP directory

I created a dedicated directory named dhcp inside the assignment folder and moved into it. This directory will store all the configuration files required to build the DHCP server container image.

```
root@VCCT:/home/sawera# mkdir -p ~/assignment02/dhcp && cd ~/assignment02/dhcp
root@VCCT:~/assignment02/dhcp#
```

#### 02. Writing the Dockerfile

A new Dockerfile was created with the base image ubuntu:24.04. I added commands to install the isc-dhcp-server package, remove cached files, and copy the DHCP configuration file into the container.

```
root@VCCT:~/assignment02/dhcp# cat > Dockerfile <<'EOF'
FROM ubuntu:24.04
> RUN apt-get update && apt-get install -y isc-dhcp-server && rm -rf /var/lib/ap
t/lists/*
> COPY dhcpd.conf /etc/dhcp/dhcpd.conf
> RUN mkdir -p /var/lib/dhcp
> EXPOSE 67/udp
> CMD ["dhcpd","-4","-f","-d","--no-pid"]
> EOF
```

#### 03. Verifying Dockerfile content

The completed Dockerfile exposes UDP port 67 and defines the default command to start the DHCP daemon (dhcpd -4 -f -d --no-pid), which enables it to run in the foreground for container logging and debugging.

```
root@VCCT:~/assignment02/dhcp# cat Dockerfile
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y isc-dhcp-server && rm -rf /var/lib/apt/
lists/*
COPY dhcpd.conf /etc/dhcp/dhcpd.conf
RUN mkdir -p /var/lib/dhcp
EXPOSE 67/udp
CMD ["dhcpd","-4","-f","-d","--no-pid"]
root@VCCT:~/assignment02/dhcp#
```

#### 04. Creating dhcpd.conf file

DHCP configuration file defining parameters such as the lease duration, subnet range, gateway, subnet mask, and DNS server. This ensures that clients automatically receive IP addresses within 192.168.8.130 – 192.168.8.160.

```
root@VCCT:~/assignment02/dhcp# cat > dhcpd.conf <<'EOF'
> authoritative;
> default-lease-time 43200;    # 12h
> max-lease-time      86400;    # 24h
>
> subnet 192.168.8.0 netmask 255.255.255.0 {
>     range 192.168.8.130 192.168.8.160;
>     option routers 192.168.8.1;
>     option subnet-mask 255.255.255.0;
>     option domain-name "csne.vcct.com";
>     option domain-name-servers 192.168.8.132;
>     allow unknown-clients;
> }
> EOF
root@VCCT:~/assignment02/dhcp#
```

## 05. Verifying dhcpd.conf configuration

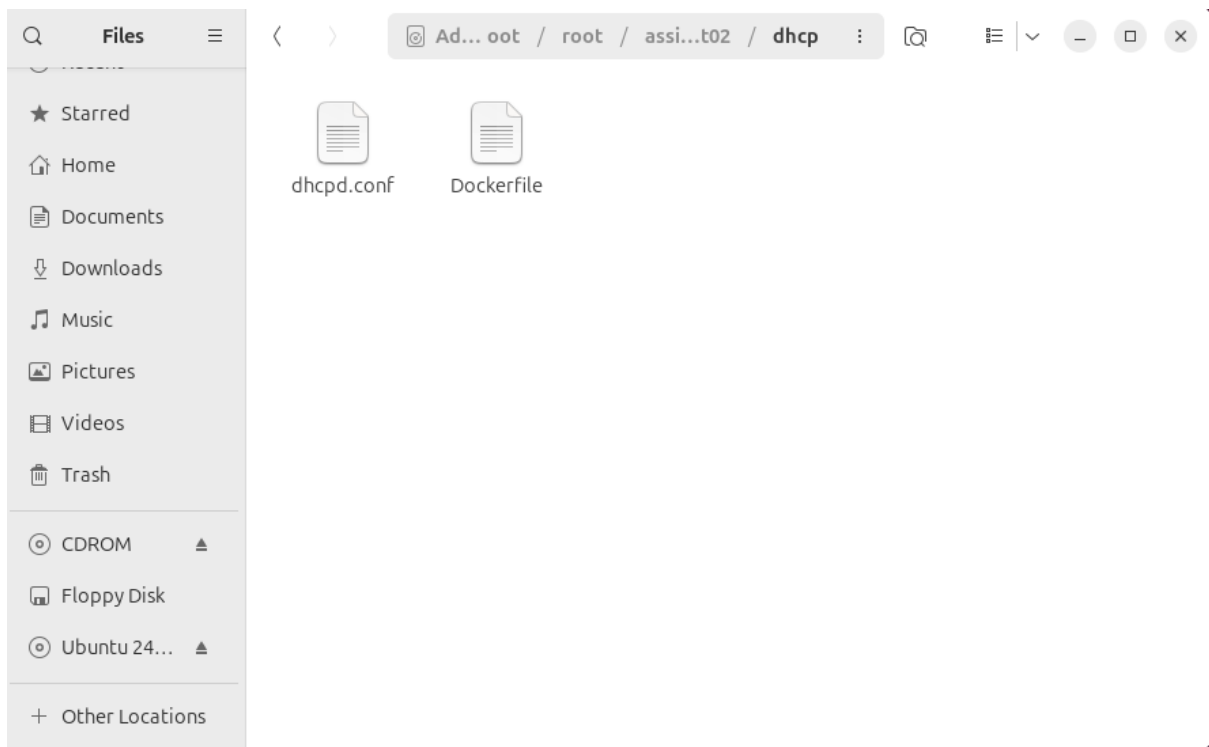
Reviewed the configuration to confirm that it includes essential network details like the router IP (192.168.8.1) and the domain name (csne.vcct.com). The setup also allows unknown clients to obtain IP leases.

```
root@VCCT:~/assignment02/dhcp# cat dhcpd.conf
authoritative;
default-lease-time 43200;    # 12h
max-lease-time      86400;    # 24h

subnet 192.168.8.0 netmask 255.255.255.0 {
    range 192.168.8.130 192.168.8.160;
    option routers 192.168.8.1;
    option subnet-mask 255.255.255.0;
    option domain-name "csne.vcct.com";
    option domain-name-servers 192.168.8.132;
    allow unknown-clients;
}
root@VCCT:~/assignment02/dhcp#
```

## 06. Folder verification

Both the Dockerfile and dhcpd.conf are present in the DHCP folder, confirming the environment is ready for building the image.



## 5.2 Building the DHCP Image

built the image using the command:

```
docker build -t sawera2001/dhcp-isc:1.0 .
```

The build process completed successfully, packaging the DHCP server and configuration files into a reusable container image.

```
root@VCCT:~/assignment02/dhcp# docker build -t sawera2001/dhcp-isc:1.0 .
[+] Building 34.2s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 263B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:24.04    3.7s
=> [auth] library/ubuntu:pull token for registry-1.docker.io      0.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> CACHED [1/4] FROM docker.io/library/ubuntu:24.04@sha256:353675e2a41ba 0.0s
=> [internal] load build context                                  0.1s
=> => transferring context: 390B                                      0.0s
=> [2/4] RUN apt-get update && apt-get install -y isc-dhcp-server && rm 29.0s
=> [3/4] COPY dhcpd.conf /etc/dhcp/dhcpd.conf                    0.2s
=> [4/4] RUN mkdir -p /var/lib/dhcp                              0.8s
=> exporting to image                                              0.2s
=> => exporting layers                                              0.2s
=> => writing image sha256:3f98137fbb7903c9fb7ec288a2a3ba7390b5c73d29927 0.0s
=> => naming to docker.io/sawera2001/dhcp-isc:1.0                0.0s
root@VCCT:~/assignment02/dhcp#
```

## 5.3 Running the DHCP Container

### 01. Verifying dhcpd.conf configuration

Run the DHCP container with network privileges (`--network host --cap-add=NET_ADMIN`) so that it could bind to port 67 and manage IP allocation on the host network.

```
root@VCCT:~/assignment02/dhcp# docker rm -f dhcp 2>/dev/null || true; docker run
-d --name dhcp --network host --cap-add=NET_ADMIN -v /etc/localtime:/etc/localt
ime:ro -v /var/lib/dhcp:/var/lib/dhcp sawera2001/dhcp-isc:1.0 dhcpd -4 -f -d --n
o-pid -cf /etc/dhcp/dhcpd.conf -lf /var/lib/dhcp/dhcpd.leases ens37

dhcp
a1ee13d50b6f48e0259cf3d113ab42612461c54a6ad9096c6386b61819bb7743
root@VCCT:~/assignment02/dhcp#
root@VCCT:~/assignment02/dhcp#
```

### 02. Checking running containers

The output of `docker ps` shows that the DHCP container is running using the image `sawera2001/dhcp-isc:1.0`, confirming the server started successfully.

```
root@VCCT:~/assignment02/dhcp# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS   NAMES
2ef5b0939a4c   sawera2001/dhcp-isc:1.0            "dhcpd -4 -f -d --no..." About a minute ago Up 59 seconds      dhcp
root@VCCT:~/assignment02/dhcp# docker logs -f dhcp
```

### 03. Viewing DHCP logs

Using `docker logs -f dhcp`, I monitored the container logs and verified that the DHCP service was active. The log indicates it is listening on interface `ens37` for the `192.168.8.0/24` network and has successfully started the service.

```
root@VCCT:~/assignment02/dhcp# docker logs -f dhcp

Internet Systems Consortium DHCP Server 4.4.3-P1
Copyright 2004-2022 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/
Config file: /etc/dhcp/dhcpd.conf
Database file: /var/lib/dhcp/dhcpd.leases
PID file: /var/run/dhcpd.pid
Wrote 0 leases to leases file.
Listening on LPF/ens37/00:0c:29:1b:42:4b/192.168.8.0/24
Sending on   LPF/ens37/00:0c:29:1b:42:4b/192.168.8.0/24
Sending on   Socket/fallback/fallback-net
Server starting service.
```

## 5.4 Testing DHCP Functionality

```
root@VCCT:~/assignment02/dhcp# docker logs -f dhcp

Internet Systems Consortium DHCP Server 4.4.3-P1
Copyright 2004-2022 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/
Config file: /etc/dhcp/dhcpd.conf
Database file: /var/lib/dhcp/dhcpd.leases
PID file: /var/run/dhcpd.pid
Wrote 0 leases to leases file.
Listening on LPF/ens37/00:0c:29:1b:42:4b/192.168.8.0/24
Sending on   LPF/ens37/00:0c:29:1b:42:4b/192.168.8.0/24
Sending on   Socket/fallback/fallback-net
Server starting service.
```

## 06. Part D – Proxy Server (Squid)

### 6.1 Dockerfile Creation

#### 01. Creating the Squid directory

Creating a new folder named squid under the assignment directory and switched into it. This directory is used to store the Squid proxy configuration and Dockerfile.

```
root@VCCT:~/assignment02# mkdir -p ~/assignment02/squid && cd ~/assignment02/squid
root@VCCT:~/assignment02/squid#
```

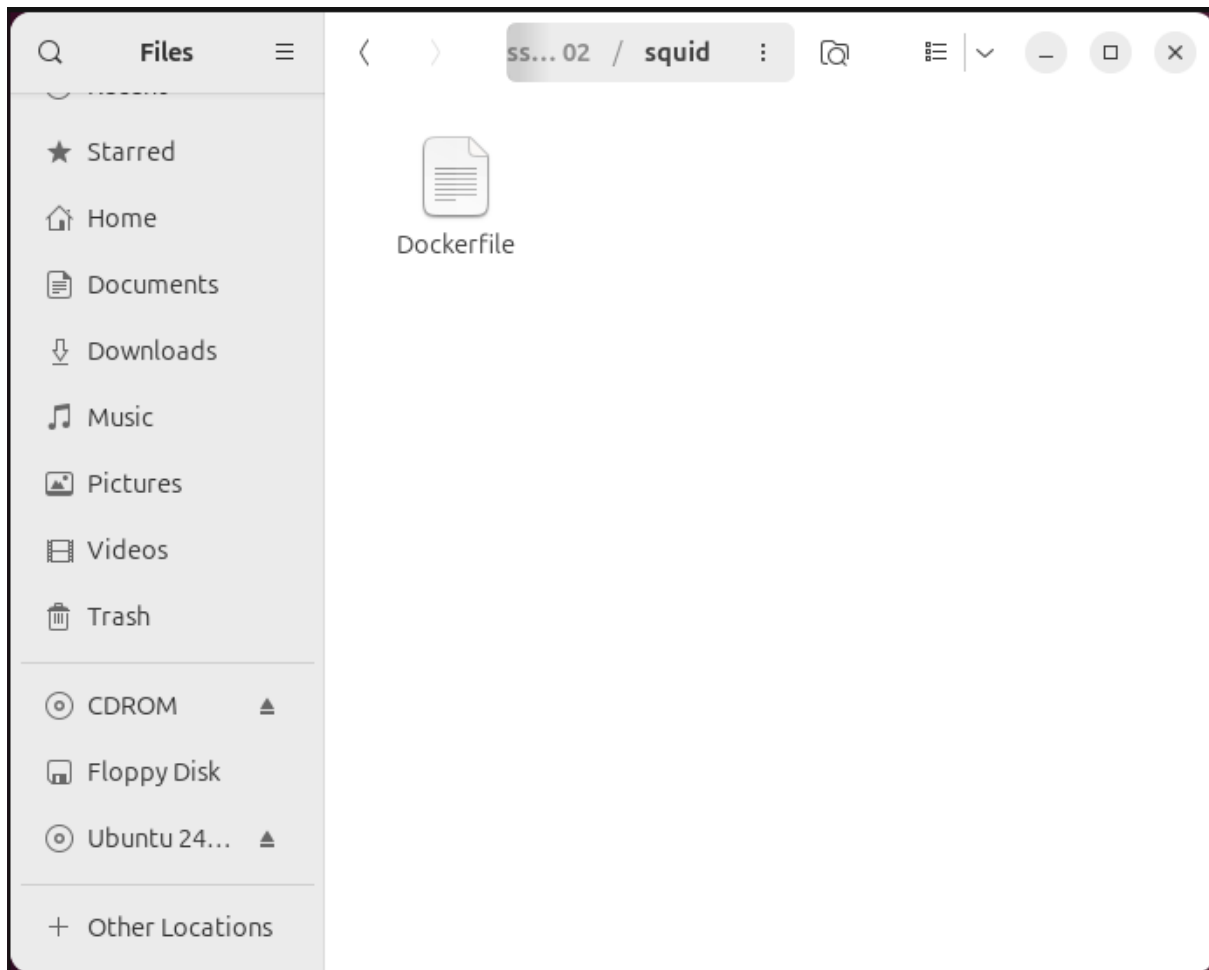
#### 02. Writing the Dockerfile

Created a Dockerfile using the ubuntu:24.04 base image. It installs the Squid proxy package, removes temporary files, and copies the local Squid configuration file into /etc/squid/squid.conf.

```
root@VCCT:~/assignment02/squid#
root@VCCT:~/assignment02/squid# cat > Dockerfile <<'EOF'
>
> FROM ubuntu:24.04
>
> RUN apt-get update && apt-get install -y squid && rm -rf /var/lib/apt/lists/*
>
> COPY squid.conf /etc/squid/squid.conf
>
> RUN mkdir -p /var/spool/squid && chown -R proxy:proxy /var/spool/squid
>
> EXPOSE 3128
>
> CMD ["squid", "-N", "-d", "1"]
>
> EOF
root@VCCT:~/assignment02/squid#
```

#### 03. Verifying the Dockerfile file creation

After saving the file, I verified that it appears correctly inside the folder. This ensures that the Docker build context is properly prepared.



#### 04. Creating the Squid configuration file

Wrote the main Squid configuration file (squid.conf) to define access control, caching policy, and logging locations. The configuration allows access only from the local 192.168.8.0/24 subnet, sets up port 3128 for HTTP proxying, defines cache sizes, and specifies log locations for monitoring proxy activity.

```
root@VCCT:~/assignment02/squid# cat > squid.conf <<'EOF'
>
> # Port
>
http> http_port 3128
>
>
>
> # Allow only local subnet
>
> acl localnet src 192.168.8.0/24
>
> http_access allow localnet
>
> http_access deny all
>
>
>
> # Cache tuning (100 MB on disk)
>
> cache_mem 64 MB
>
> maximum_object_size_in_memory 1 MB
>
> cache_dir ufs /var/spool/squid 100 16 256
>
>
>
> # Logs
>
> access_log /var/log/squid/access.log
>
> cache_log /var/log/squid/cache.log
>
```

```
>
> # Logs
>
> access_log /var/log/squid/access.log
>
> cache_log /var/log/squid/cache.log
>
>
>
> cache_effective_user proxy
>
> visible_hostname squid-proxy
>
> dns_v4_first on
>
> EOF
```

## 6.2 Building the Proxy Image

### 01. Building the image with Docker

Built the Squid proxy image using the command:

```
docker build -t sawera2001/squid-proxy:1.0 .
```

The build completed successfully after installing Squid and copying the configuration files into the image.

```
root@VCCT:~/assignment02/squid# docker build -t sawera2001/squid-proxy:1.0 .

[+] Building 316.6s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                 0.1s
=> => transferring dockerfile: 291B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:24.04     7.5s
=> [auth] library/ubuntu:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                    0.0s
=> => transferring context: 2B                                       0.0s
=> [1/4] FROM docker.io/library/ubuntu:24.04@sha256:728785b59223  75.0s
=> => resolve docker.io/library/ubuntu:24.04@sha256:728785b59223d 0.0s
=> => sha256:ce8f79aecc435fc0b22d4dd58c72836e330b 2.30kB / 2.30kB 0.0s
=> => sha256:a1a21c96bc16121569dd937bcd1c745a5 29.72MB / 29.72MB 67.6s
=> => sha256:728785b59223d755e3e5c5af178fab1be703 6.69kB / 6.69kB 0.0s
=> => sha256:78281ac7684a7caf02348780a1b5de85844548a3 424B / 424B 0.0s
=> => extracting sha256:a1a21c96bc16121569dd937bcd1c745a5081629b3 6.7s
=> [internal] load build context                                    0.1s
=> => transferring context: 468B                                     0.0s
=> [2/4] RUN apt-get update && apt-get install -y squid && rm - 228.9s
=> [3/4] COPY squid.conf /etc/squid/squid.conf                    0.1s
=> [4/4] RUN mkdir -p /var/spool/squid && chown -R proxy:proxy /v 1.1s
=> exporting to image                                              3.7s
=> => exporting layers                                              3.7s
=> => writing image sha256:cff0cc90835a2bb8bdad6612dda38513c0b407 0.0s
=> => naming to docker.io/sawera2001/squid-proxy:1.0              0.0s
root@VCCT:~/assignment02/squid#
```

## 6.3 Running the Proxy Container

### 01. Running the Squid container

The container was started using the command:

```
docker run -d --name squid --network host --cpus="1.0" --memory="512m"
sawera2001/squid-proxy:1.1
```

This runs the proxy server with resource limits and host network access.

```

root@VCCT:~/assignment02/squid# docker rm -f squid 2>/dev/null || true
; docker run -d --name squid --network host --cpus="1.0" --memory="512
m" sawera2001/squid-proxy:1.1

squid
655b92bd6dfb6c46ea8bc2a6b44f2a82573dafecfaa3586b834091acd6fcf070

```

## 01. Verifying container status

Using docker ps, I confirmed that the Squid container is active and listening on port 3128 along with the other containers (DNS and DHCP).

```

root@VCCT:~/assignment02/squid# docker ps

```

| CONTAINER ID | IMAGE                      | COMMAND                  | CREATED        | STATUS        | PORTS | NAMES |
|--------------|----------------------------|--------------------------|----------------|---------------|-------|-------|
| 655b92bd6dfb | sawera2001/squid-proxy:1.1 | "squid -N -d 1"          | 10 seconds ago | Up 10 seconds |       | squid |
| 91cc28fc58e6 | sawera2001/dns-bind9:1.0   | "named -g -c /etc/bi..." | 2 hours ago    | Up 2 hours    |       | dns   |
| 2ef5b0939a4c | sawera2001/dhcp-isc:1.0    | "dhcpd -4 -f -d --no..." | 4 hours ago    | Up 4 hours    |       | dhcp  |

```

root@VCCT:~/assignment02/squid#

```

## 6.4 Testing Proxy and Caching

### 01. Testing web access via proxy

used the curl command to send HTTP requests through the Squid proxy:

```

root@VCCT:~/assignment02/squid# curl -I -x 192.168.8.132:3128 http://example.com/

curl -I -x 192.168.8.132:3128 http://example.com/

curl -I -x 192.168.8.132:3128 http://ubuntu.com/

curl -I -x 192.168.8.132:3128 http://ubuntu.com/

HTTP/1.1 200 OK
Content-Type: text/html
ETag: "84238dfc8092e5d9c0dac8ef93371a07:1736799080.121134"
Last-Modified: Mon, 13 Jan 2025 20:11:20 GMT
Cache-Control: max-age=86000
Date: Sat, 04 Oct 2025 14:00:48 GMT
Cache-Status: squid-proxy;detail=mismatch
Via: 1.1 squid-proxy (squid/6.13)
Connection: keep-alive

root@VCCT:~/assignment02/squid#
root@VCCT:~/assignment02/squid# curl -I -x 192.168.8.132:3128 http://example.com/
HTTP/1.1 200 OK
Content-Type: text/html
ETag: "84238dfc8092e5d9c0dac8ef93371a07:1736799080.121134"
Last-Modified: Mon, 13 Jan 2025 20:11:20 GMT
Cache-Control: max-age=86000
Date: Sat, 04 Oct 2025 14:00:48 GMT
Age: 0
Cache-Status: squid-proxy;hit;detail=match
Via: 1.1 squid-proxy (squid/6.13)
Connection: keep-alive

root@VCCT:~/assignment02/squid#
root@VCCT:~/assignment02/squid# curl -I -x 192.168.8.132:3128 http://ubuntu.com/

```

The response shows HTTP status 200 OK, proving that the proxy is successfully handling web requests.

## 07. Part E – Docker Hub

### 7.1 Pushing Images to Docker Hub

#### 01. Logging in to Docker Hub

logged in to my Docker Hub account using the docker login command. Authentication succeeded with my Docker ID **sawera2001**, allowing me to push and pull container images from my personal repository

```
root@VCCT:~/assignment02# docker login

Authenticating with existing credentials... [Username: sawera2001]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
```

#### 02. Listing local images

Before pushing, I checked all locally built images using docker images. It showed three custom images:

- sawera2001/dns-bind9:1.0 (DNS server)
- sawera2001/dhcp-isc:1.0 (DHCP server)
- sawera2001/squid-proxy:1.1 (Proxy server)

```
root@VCCT:~/assignment02# docker images | egrep 'sawera2001/(dns-bind9|dhcp-isc|squid-proxy)'
```

|                        |     |              |                |        |
|------------------------|-----|--------------|----------------|--------|
| sawera2001/squid-proxy | 1.0 | 7518a96f9031 | 37 minutes ago | 262 MB |
| sawera2001/squid-proxy | 1.1 | 0796dd379acd | 37 minutes ago | 262 MB |
| sawera2001/dhcp-isc    | 1.0 | 3f98137fbb79 | 10 days ago    | 83.6MB |
| sawera2001/dns-bind9   | 1.0 | 1d9c63fe6f83 | 10 days ago    | 131 MB |

### 03. Pushing images to Docker Hub

Each image was uploaded to Docker Hub using the docker push command. I pushed all three service containers sequentially. The successful push messages confirmed that the images were stored on the Docker Hub registry under my username **sawera2001**.

```
root@VCCT:~/assignment02# docker push sawera2001/dns-bind9:1.0
docker The push refers to repository [docker.io/sawera2001/dns-bind9]
push sawera2001/dhcp-isc:1.0

docker push sawera2001/squid-proxy:1.1
5dc5aa613025: Pushed
bba078e6490b: Pushed
ee4ace259b3b: Pushed
8eb7b1375d72: Pushed
f9f52dc133e2: Mounted from library/ubuntu
1.0: digest: sha256:ad53ff13c9d20e89aeb7d47a550d152d06f4e27de108717098841c6d5f8fad9e size: 1362
root@VCCT:~/assignment02#
root@VCCT:~/assignment02# docker push sawera2001/dhcp-isc:1.0
The push refers to repository [docker.io/sawera2001/dhcp-isc]
5f70bf18a086: Pushed
0afd3f27c2d1: Pushed
7bc943ddec0b: Pushed
f9f52dc133e2: Mounted from sawera2001/dns-bind9
1.0: digest: sha256:6773544382af5f8e84c6223c05d896650c43c0a35c1ecff62a7dc31029d77041 size: 1153
root@VCCT:~/assignment02#
root@VCCT:~/assignment02# docker push sawera2001/squid-proxy:1.1
The push refers to repository [docker.io/sawera2001/squid-proxy]
627d87f662d9: Pushed
cea1b25bbda5: Pushed
304354286be0: Pushed
8a3e1c60d565: Pushed
65b08cd99c60: Mounted from library/ubuntu
1.1: digest: sha256:84de4fc5fbaca668b167f7d01eb713deef24fc8673560eceedbcaca4d7c3fd5b size: 1364
root@VCCT:~/assignment02#
```

## 7.2 Pulling Images and Redeployment

### 01. Removing and re-pulling DNS image

verify redeployment, I first removed the local DNS image using `docker rmi -f`. Then I pulled it again from Docker Hub with `docker pull sawera2001/dns-bind9:1.0`. After pulling, I ran the container successfully to confirm the image integrity and functionality.

```
root@VCCT:~/assignment02# docker rmi -f sawera2001/dns-bind9:1.0 2>/dev/null || true

docker pull sawera2001/dns-bind9:1.0

docker run -d Untagged: sawera2001/dns-bind9:1.0
Untagged: sawera2001/dns-bind9@sha256:ad53ff13c9d20e89aeb7d47a550d152d06f4e27de108717098841c6d5f8fad9e
Deleted: sha256:1d9c63fe6f83f01315ef8fe8d3607d8f3f6f5e123b31a282ba4c3ef92628c2b5
--root@VCCT:~/assignment02#
root@VCCT:~/assignment02# docker pull sawera2001/dns-bind9:1.0
name dns --network host --restart unless-stopped sawera2001/dns-bind9:1.0
csne.vcct.com A

1.0: Pulling from sawera2001/dns-bind9
953cdd413371: Already exists
5caa112519db: Already exists
c0f9195963de: Already exists
8ec87291cc52: Already exists
56c821d38dfa: Already exists
Digest: sha256:ad53ff13c9d20e89aeb7d47a550d152d06f4e27de108717098841c6d5f8fad9e
Status: Downloaded newer image for sawera2001/dns-bind9:1.0
docker.io/sawera2001/dns-bind9:1.0
root@VCCT:~/assignment02#
root@VCCT:~/assignment02# docker run -d --name dns --network host --restart unless-stopped sawera2001/dns-bind9:1.0
8236a42bf3ae5ab49970cc9817a1a9a04394b4ab8d6b003ebb3a80d93867c82f
```

### 02. Redeploying DHCP server

Similarly, pulled the DHCP image (`sawera2001/dhcp-isc:1.0`) from Docker Hub and ran it using `docker run` with `host-network` and `NET_ADMIN` capabilities. The log output confirmed that the DHCP server started successfully and was serving the `192.168.8.0/24` network.

```
root@VCCT:~/assignment02# docker rm -f dhcp 2>/dev/null || true

dhcp
root@VCCT:~/assignment02#
root@VCCT:~/assignment02# docker pull sawera2001/dhcp-isc:1.0

1.0: Pulling from sawera2001/dhcp-isc
Digest: sha256:6773544382af5f8e84c6223c05d896650c43c0a35c1ecff62a7dc31029d77041
Status: Image is up to date for sawera2001/dhcp-isc:1.0
docker.io/sawera2001/dhcp-isc:1.0
root@VCCT:~/assignment02#
root@VCCT:~/assignment02# docker run -d --name dhcp --network host --cap-add=NET_ADMIN -v /var/lib/dhcp:/var/lib/dhcp sawera2001/dhcp-isc:1.0 dhcpd -4 -f -d --no-p
id -cf /etc/dhcp/dhcpd.conf -lf /var/lib/dhcp/dhcpd.leases ens37
fa7ebfec689a94355a270f90eff40457d8de2fb79ddff87d78ffe420c0907281
root@VCCT:~/assignment02#
root@VCCT:~/assignment02# docker logs -f dhcp | sed -n '1,30p'

Internet Systems Consortium DHCP Server 4.4.3-P1
Copyright 2004-2022 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/
Config file: /etc/dhcp/dhcpd.conf
Database file: /var/lib/dhcp/dhcpd.leases
PID file: /var/run/dhcpd.pid
Wrote 0 leases to leases file.
Listening on LPF/ens37/00:0c:29:1b:42:4b/192.168.8.0/24
Sending on LPF/ens37/00:0c:29:1b:42:4b/192.168.8.0/24
Sending on Socket/fallback/Fallback-net
Server starting service.
```

### 03. Redeploying Proxy server

Repeated the process for the Squid proxy image (sawera2001/squid-proxy:1.1). After pulling, I started the container and verified that all three services (DNS, DHCP, and Proxy) were running together using docker ps.

```
root@VCCT:~/assignment02# docker rmi -f sawera2001/squid-proxy:1.1 2>/dev/null || true
Untagged: sawera2001/squid-proxy:1.1
Untagged: sawera2001/squid-proxy@sha256:84de4fc5fbaca668b167f7d01eb713deef24fc8673560eceedbcaca4d7c3fd5b
Deleted: sha256:0796dd379acd56ebfb5aa70c5439d5bdc5494f39d34e76aa68a18e8a3bcd8c10
root@VCCT:~/assignment02#
root@VCCT:~/assignment02# docker pull sawera2001/squid-proxy:1.1

1.1: Pulling from sawera2001/squid-proxy
a1a21c96bc16: Already exists
cc457bd5e13f: Already exists
67a08f340d2a: Already exists
85062eb6ad77: Already exists
04956cbfcbcb: Already exists
Digest: sha256:84de4fc5fbaca668b167f7d01eb713deef24fc8673560eceedbcaca4d7c3fd5b
Status: Downloaded newer image for sawera2001/squid-proxy:1.1
docker.io/sawera2001/squid-proxy:1.1
root@VCCT:~/assignment02#
root@VCCT:~/assignment02# docker run -d --name squid --network host --cpus="1.0" --memory="512m" sawera2001/squid-proxy:1.1
f69f669a559fa1a84363c1019acbf9696e777eda8bf9774e8f4bb659e9a57cb
root@VCCT:~/assignment02#
root@VCCT:~/assignment02# docker ps
```

## 08. Part F – Verification

### 8.1 Using docker ps to Verify Services

This step confirms that all required containers — **DHCP**, **DNS**, and **Proxy (Squid)** — are running correctly on the Docker host.

The `docker ps` command lists all active containers along with their **Container IDs**, **image names**, **status**, and **uptime**.

As shown in the screenshot, the containers named `dhcp`, `dns`, and `squid` are all running successfully, each built from the corresponding Docker images under the **sawera2001** repository

```
root@VCCT:~/assignment02# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES
f69f669a559f   sawera2001/squid-proxy:1.1         "squid -N -d 1"         About a minute Up About a minute           squid
fa7ebfec689a   sawera2001/dhcp-isc:1.0            "dhcpd -4 -f -d --no..." 38 minutes ago Up 38 minutes           dhcp
8236a42bf3ae   sawera2001/dns-bind9:1.0          "named -g -c /etc/bi..."  About an hour ago Up About an hour           dns
root@VCCT:~/assignment02#
```

### 8.1 Final Validation

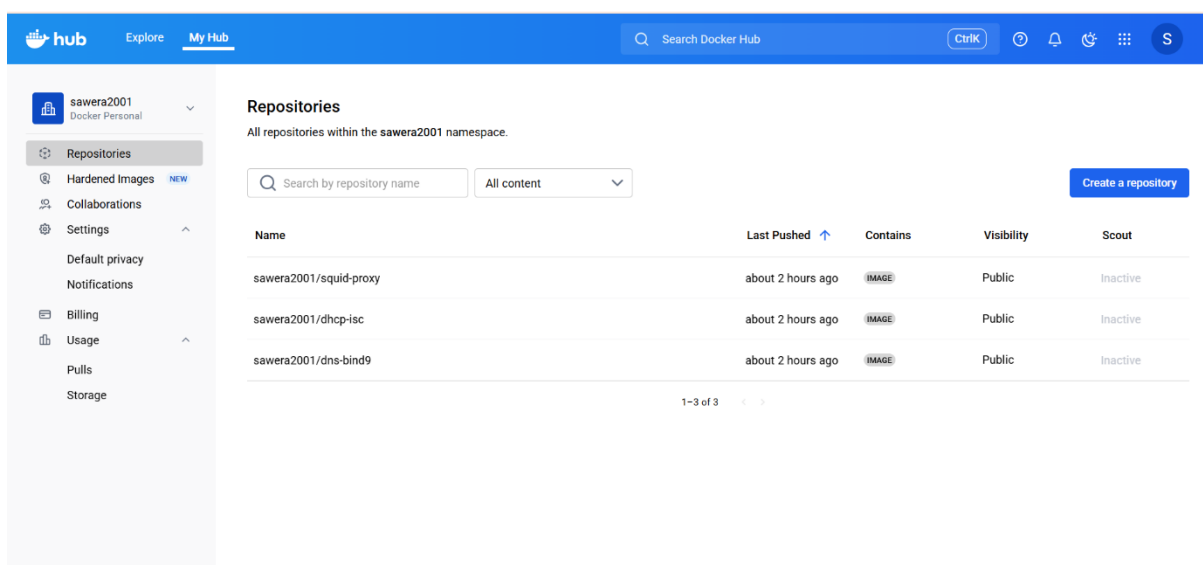
Finally, the **Docker Hub** dashboard was checked to ensure that all images were successfully uploaded and publicly available.

The screenshot shows the **sawera2001 Docker Hub repositories**, containing:

- sawera2001/squid-proxy
- sawera2001/dhcp-isc
- sawera2001/dns-bind9

Each repository is marked **Public**, confirming that the images are accessible for redeployment and sharing.

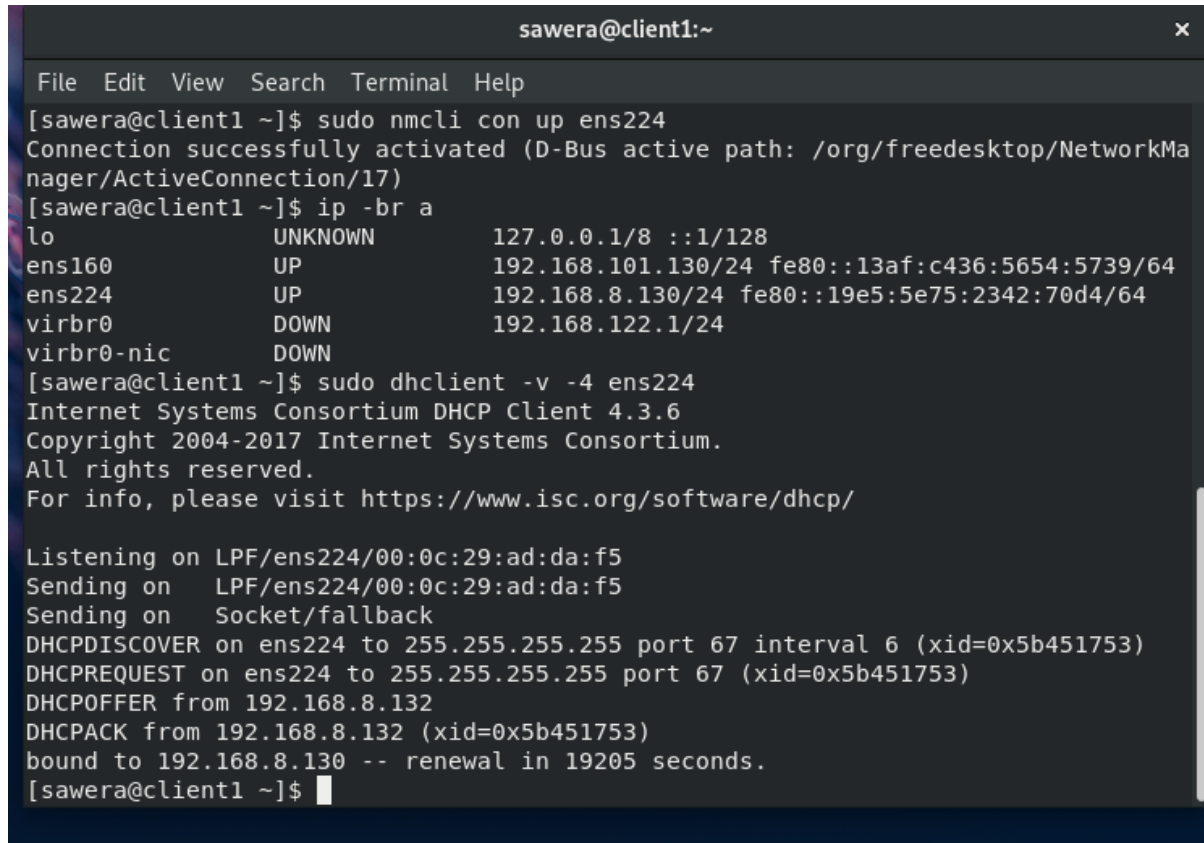
This verifies the completion of all previous parts, proving that the full Dockerized network service environment (DHCP, DNS, and Squid Proxy) was correctly built, uploaded, and validated



## 09. Client Testing and Verification

### 9.1 DHCP Client Configuration

The client connected to the network interface ens224 and requested an IP address via DHCP. The DHCP container running on the server responded successfully, assigning IP 192.168.8.130 to the client. The DHCP handshake process (DISCOVER → OFFER → REQUEST → ACK) was completed, confirming the DHCP server's functionality.



```
sawera@client1:~  
File Edit View Search Terminal Help  
[sawera@client1 ~]$ sudo nmcli con up ens224  
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkMa  
nager/ActiveConnection/17)  
[sawera@client1 ~]$ ip -br a  
lo UNKNOWN 127.0.0.1/8 ::1/128  
ens160 UP 192.168.101.130/24 fe80::13af:c436:5654:5739/64  
ens224 UP 192.168.8.130/24 fe80::19e5:5e75:2342:70d4/64  
virbr0 DOWN 192.168.122.1/24  
virbr0-nic DOWN  
[sawera@client1 ~]$ sudo dhclient -v -4 ens224  
Internet Systems Consortium DHCP Client 4.3.6  
Copyright 2004-2017 Internet Systems Consortium.  
All rights reserved.  
For info, please visit https://www.isc.org/software/dhcp/  
  
Listening on LPF/ens224/00:0c:29:ad:da:f5  
Sending on LPF/ens224/00:0c:29:ad:da:f5  
Sending on Socket/fallback  
DHCPDISCOVER on ens224 to 255.255.255.255 port 67 interval 6 (xid=0x5b451753)  
DHCPREQUEST on ens224 to 255.255.255.255 port 67 (xid=0x5b451753)  
DHCPOFFER from 192.168.8.132  
DHCPACK from 192.168.8.132 (xid=0x5b451753)  
bound to 192.168.8.130 -- renewal in 19205 seconds.  
[sawera@client1 ~]$
```

### 9.2 DNS Resolution Test

Using the `dig @192.168.8.132 csne.vcct.com` command, the client verified name resolution through the DNS container. The domain `csne.vcct.com` was successfully resolved to IP 192.168.8.132. This confirms that the DNS (BIND9) service is active and reachable.

```
sawera@client1:~  
File Edit View Search Terminal Help  
; <<>> DiG 9.11.10-RedHat-9.11.10-1.fc29 <<>> @192.168.8.132 csne.vcct.com  
; (1 server found)  
;; global options: +cmd  
;; Got answer:  
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 28692  
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1  
;; WARNING: recursion requested but not available  
  
;; OPT PSEUDOSECTION:  
; EDNS: version: 0, flags:; udp: 1232  
; COOKIE: d419b155674da4c80100000068e20ddd6bd79356e0f26c4f (good)  
;; QUESTION SECTION:  
;csne.vcct.com.                IN      A  
  
;; ANSWER SECTION:  
csne.vcct.com.                3600    IN      A      192.168.8.132  
  
;; Query time: 1 msec  
;; SERVER: 192.168.8.132#53(192.168.8.132)  
;; WHEN: Sun Oct 05 11:49:08 +0530 2025  
;; MSG SIZE rcvd: 86  
[sawera@client1 ~]$
```

### 9.3 Proxy Server Functionality

The proxy server was tested using the `curl -I -x 192.168.8.132:3128` command to access `example.com` and `ubuntu.com`. The HTTP response headers (200 OK and 301 Moved Permanently) indicate successful web traffic routing via the Squid proxy container. The “Via: squid-proxy (squid/6.13)” header confirms proper proxy handling and caching.

```
sawera@client1:~  
File Edit View Search Terminal Help  
--- 192.168.8.132 ping statistics ---  
13 packets transmitted, 13 received, 0% packet loss, time 44ms  
rtt min/avg/max/mdev = 1.141/2.005/4.301/0.988 ms  
[sawera@client1 ~]$ curl -I -x 192.168.8.132:3128 http://example.com  
HTTP/1.1 200 OK  
Content-Type: text/html  
ETag: "84238dfc8092e5d9c0dac8ef93371a07:1736799080.121134"  
Last-Modified: Mon, 13 Jan 2025 20:11:20 GMT  
Cache-Control: max-age=86000  
Date: Sun, 05 Oct 2025 06:24:36 GMT  
Cache-Status: squid-proxy;detail=mismatch  
Via: 1.1 squid-proxy (squid/6.13)  
Connection: keep-alive  
  
[sawera@client1 ~]$ curl -I -x 192.168.8.132:3128 http://ubuntu.com  
HTTP/1.1 301 Moved Permanently  
Content-Length: 0  
Location: https://ubuntu.com/  
Date: Sun, 05 Oct 2025 06:25:32 GMT  
Cache-Status: squid-proxy;detail=mismatch  
Via: 1.1 squid-proxy (squid/6.13)  
Connection: keep-alive  
[sawera@client1 ~]$
```

### **9.3 Summary of Client-Side Validation**

All services performed as expected:

- DHCP successfully allocated an IP to the client.
- DNS resolved hostnames accurately.
- Squid proxy served and cached HTTP requests.

This confirms end-to-end communication between the client and Docker-based infrastructure.

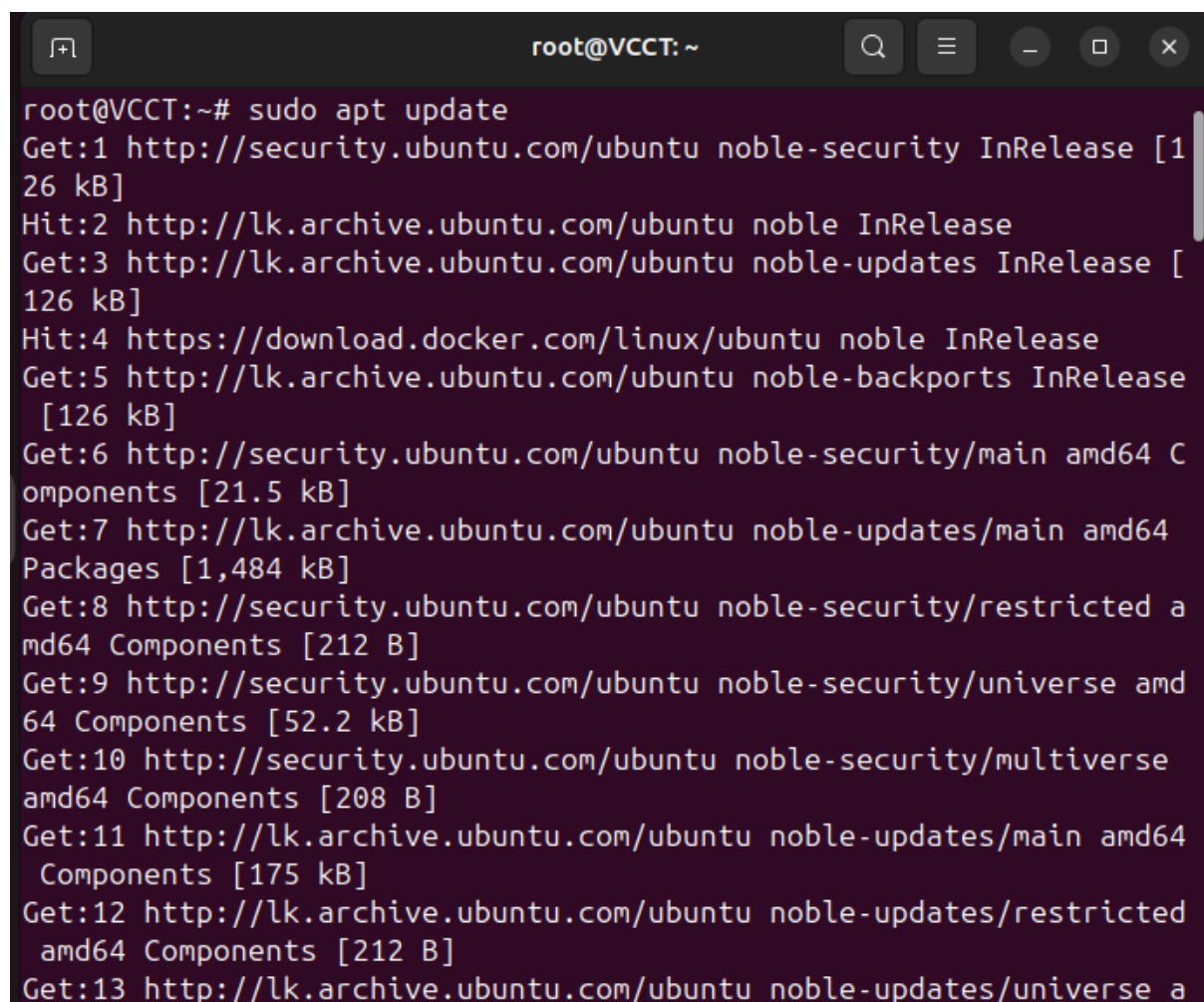
## 10. Bonus Task – Docker Compose

This task demonstrates how all three core services (DNS, DHCP, and Proxy) were integrated and managed using Docker Compose. The objective was to simplify multi-container orchestration by defining the services within a single docker-compose.yml file and deploying them together.

### 10.1 Pulling Images and Redeployment

#### 01. Updating System Packages

The process began with updating the system repositories using the `sudo apt update` command. This ensured all Ubuntu packages and dependencies were up to date before installing Docker Compose.

A terminal window titled 'root@VCCT: ~' with standard Ubuntu window controls. The terminal displays the output of the 'sudo apt update' command. It shows the process of updating various Ubuntu repositories, including security, main, universe, and multiverse components for noble, noble-updates, and noble-backports. The output includes file sizes for each component and package list sizes. The terminal text is as follows:

```
root@VCCT:~# sudo apt update
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://lk.archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://lk.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:4 https://download.docker.com/linux/ubuntu noble InRelease
Get:5 http://lk.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:7 http://lk.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,484 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [52.2 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Get:11 http://lk.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:12 http://lk.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:13 http://lk.archive.ubuntu.com/ubuntu noble-updates/universe a
```

#### 02. Installing Docker Compose

This installed Docker Compose and its dependencies, enabling the system to run and manage multiple containers through YAML configuration.

```
root@VCCT: ~  
root@VCCT:~# sudo apt install docker-compose -y  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following package was automatically installed and is no longer  
required:  
  libllvm19  
Use 'sudo apt autoremove' to remove it.  
The following additional packages will be installed:  
  containerd.io docker-ce docker-ce-cli python3-compose  
  python3-docker python3-dockerpty python3-docopt python3-dotenv  
  python3-packaging python3-texttable python3-websocket  
Suggested packages:  
  cgroupfs-mount | cgroup-lite docker-model-plugin  
Recommended packages:  
  docker.io  
The following NEW packages will be installed:  
  docker-compose python3-compose python3-docker python3-dockerpty  
  python3-docopt python3-dotenv python3-packaging  
  python3-texttable python3-websocket  
The following packages will be upgraded:  
  containerd.io docker-ce docker-ce-cli  
3 upgraded, 9 newly installed, 0 to remove and 60 not upgraded.  
Need to get 68.4 MB of archives.
```

### 03. Verifying Docker Compose Installation

After installation, I verified that Docker Compose was working correctly using:

`docker compose version`

The terminal confirmed installation with version **v2.39.4**, showing that the environment was ready for Compose deployment.

```
root@VCCT:~# docker compose version  
Docker Compose version v2.39.4  
root@VCCT:~#
```

### 04. Writing docker-compose.yml

created a **docker-compose.yml** that defines all three services (DNS, DHCP, Squid).

- **dns** uses my image sawera2001/dns-bind9:1.0, runs on the host network, and restarts unless stopped.
- **dhcp** uses sawera2001/dhcp-isc:1.0, needs NET\_ADMIN capability, mounts the leases file, and runs dhcpd in the foreground.

- **squid** uses sawera2001/squid-proxy:1.1, runs on the host network, and I set light resource limits (1 CPU / 512 MB).

```
root@VCCT:~/assignment02/compose# cat > docker-compose.yml <<'YAML'
>
> version: "3.8"
>
>
>
> services:
>
>   dns:
>
>     image: sawera2001/dns-bind9:1.0
>     container_name: dns
>     network_mode: host
e>     restart: unless-stopped
>
>
>   dhcp:
>
>     image: sawera2001/dhcp-isc:1.0
>     container_name: dhcp
>     network_mode: host
>     cap_add:
>
>       - NET_ADMIN
>
>   volumes:
```

```

    dhcpd -4 -f -d --no-pid

    -cf /etc/dhcp/dhcpd.conf

    -lf /var/lib/dhcp/dhcpd.leases

    ens37

squid:

  image: sawera2001/squid-proxy:1.1

  container_name: squid

  network_mode: host

  deploy:

    resources:

      limits:

        cpus: "1.0"

        memory: 512M

YAML
root@VCCT:~/assignment02/compose#

```

## 05. Verifying the Compose File

After completing the YAML configuration, I used the `ls` command to verify that the `docker-compose.yml` file was successfully created in the `/assignment02/compose` directory.

```

root@VCCT:~/assignment02/compose# ls
docker-compose.yml
root@VCCT:~/assignment02/compose#

```

## 06. Preparing DHCP Lease Directory

```

root@VCCT:~/assignment02/compose# sudo mkdir -p /var/lib/dhcp
root@VCCT:~/assignment02/compose# sudo touch /var/lib/dhcp/dhcp.leases

```

## 10.2 Running Services via Docker Compose

### 02. Starting Containers

The containers were launched simultaneously with the command:

```
docker compose up -d
```

Docker Compose created and started all three containers — **DNS**, **DHCP**, and **Squid Proxy** — as shown by the “Started” confirmation messages.

```
root@VCCT:~/assignment02/compose# docker compose up -d
WARN[0000] /root/assignment02/compose/docker-compose.yml: the attribute `version`
is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 3/3
 ✓ Container dns      Started           0.9s
 ✓ Container squid    Started           0.9s
 ✓ Container dhcp     Started           0.9s
```

### 02. Checking Container Status

The output listed all three containers running properly: dns, dhcp, and squid.

This confirmed that the multi-service setup was functioning as expected.

```
root@VCCT:~/assignment02/compose# docker compose ps
WARN[0000] /root/assignment02/compose/docker-compose.yml: the attribute `version`
is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE                                COMMAND                                SERVICE    CREATED
STATUS             PORTS
dhcp                sawera2001/dhcp-isc:1.0            "dhcpd -4 -f -d --no..."          dhcp       18 secon
ds ago             Up 17 seconds
dns                sawera2001/dns-bind9:1.0          "named -g -c /etc/bi..."          dns        18 secon
ds ago             Up 17 seconds
squid               sawera2001/squid-proxy:1.1        "squid -N -d 1"                     squid      18 secon
ds ago             Up 17 seconds
root@VCCT:~/assignment02/compose#
```

### 03. Viewing DHCP Service Logs

The logs displayed normal startup messages including network bindings (ens37) and the message “Server starting service,” confirming the DHCP server was successfully initialized.

```

root@VCCT:~/assignment02/compose# docker compose logs dhcp | tail -
n 20

WARN[0000] /root/assignment02/compose/docker-compose.yml: the attri
bute `version` is obsolete, it will be ignored, please remove it to
avoid potential confusion
dhcp | Internet Systems Consortium DHCP Server 4.4.3-P1
dhcp | Copyright 2004-2022 Internet Systems Consortium.
dhcp | All rights reserved.
dhcp | For info, please visit https://www.isc.org/software/dhcp/
dhcp | Config file: /etc/dhcp/dhcpd.conf
dhcp | Database file: /var/lib/dhcp/dhcpd.leases
dhcp | PID file: /var/run/dhcpd.pid
dhcp | Wrote 1 leases to leases file.
dhcp | Listening on LPF/ens37/00:0c:29:1b:42:4b/192.168.8.0/24
dhcp | Sending on   LPF/ens37/00:0c:29:1b:42:4b/192.168.8.0/24
dhcp | Sending on   Socket/fallback/fallback-net
dhcp | Server starting service.
root@VCCT:~/assianment02/compose#

```

#### 04. DNS Service Verification

The query successfully returned the IP address 192.168.8.132, confirming that the DNS container was resolving the configured domain correctly.

```

; <<>> DiG 9.18.30-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.8.132 csne
.vcct.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 20087
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL:
1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: c7a1121481db99130100000068e21e8e3ffffbe22df4039f8 (good)
;; QUESTION SECTION:
;csne.vcct.com.                IN      A

;; ANSWER SECTION:
csne.vcct.com.                 3600    IN      A      192.168.8.132

;; Query time: 2 msec
;; SERVER: 192.168.8.132#53(192.168.8.132) (UDP)
;; WHEN: Sun Oct 05 13:00:22 +0530 2025
;; MSG SIZE rcvd: 86

```

## 05. Proxy Service Verification

The response returned HTTP/1.1 200 OK with a header indicating  
Via: 1.1 squid-proxy (squid/6.13), confirming successful proxy operation and caching  
functionality.

```
root@VCCT:~/assignment02/compose# curl -I -x 192.168.8.132:3128 http://example.com

HTTP/1.1 200 OK
Content-Type: text/html
ETag: "84238dfc8092e5d9c0dac8ef93371a07:1736799080.121134"
Last-Modified: Mon, 13 Jan 2025 20:11:20 GMT
Cache-Control: max-age=86000
Date: Sun, 05 Oct 2025 07:30:58 GMT
Cache-Status: squid-proxy;detail=mismatch
Via: 1.1 squid-proxy (squid/6.13)
Connection: keep-alive
```

## 11. Conclusion

In conclusion, this assignment successfully demonstrated the setup and deployment of essential network services—DNS, DHCP, and Proxy—using Docker containers and Docker Compose. Each service was configured, built, and tested to work together within a containerized environment, ensuring proper IP allocation, domain name resolution, and web traffic management. The use of Docker Compose made it easier to manage and start all services simultaneously, improving efficiency and reducing configuration complexity. Overall, the implementation achieved all objectives, proving that containerization is an effective and reliable approach for managing and automating network infrastructure.

## 12. References

- [1] Docker, Inc., *Docker Documentation*, 2025. [Online]. Available: <https://docs.docker.com/>
- [2] Internet Systems Consortium (ISC), *ISC DHCP Server Documentation*, 2025. [Online]. Available: <https://www.isc.org/dhcp/>
- [3] Internet Systems Consortium (ISC), *BIND 9 Administrator Reference Manual*, 2025. [Online]. Available: <https://bind9.readthedocs.io/>
- [4] Squid Proxy, *Squid Official Documentation*, 2025. [Online]. Available: <https://wiki.squid-cache.org/>
- [5] Canonical Ltd., *Ubuntu Server Guide 24.04 LTS*, 2025. [Online]. Available: <https://ubuntu.com/server/docs>
- [6] Y. Pan and P. J. Lucas, “Containerization of network services using Docker and microservices architecture,” *IEEE Access*, vol. 10, pp. 13125–13134, 2022.
- [7] M. Merkel, D. Stenberg, and K. Anderson, “Practical network automation using containers and orchestration tools,” *IEEE Network*, vol. 37, no. 1, pp. 58–65, Jan.–Feb. 2023.